

Technical Architecture Specification

(V3.0)

Project: macOS Developer AI Companion (Code Refactoring, Understanding & Anywhere Explanation Engine)

Classification: Internal Engineering Design & Implementation Specification

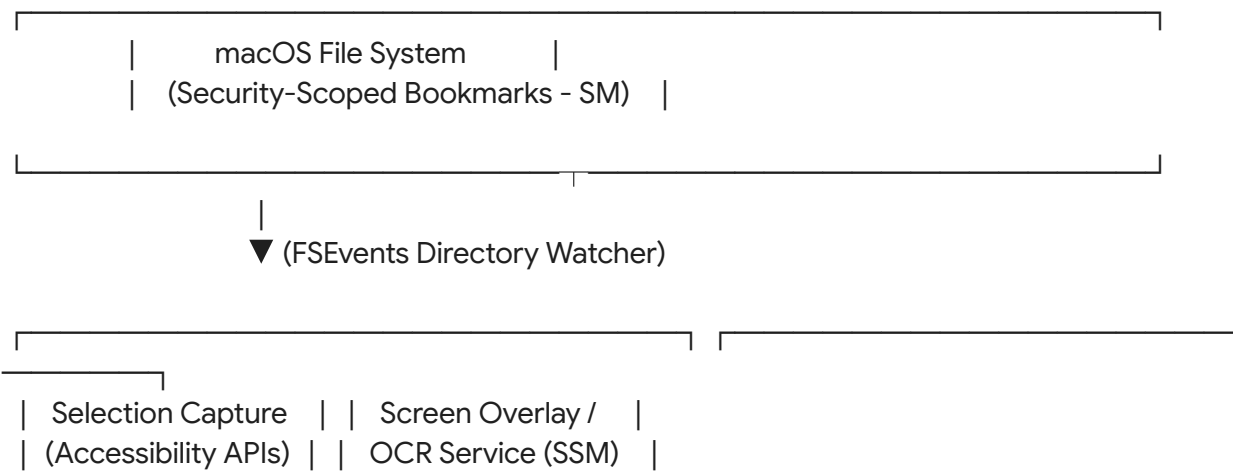
1. Executive Architecture Overview

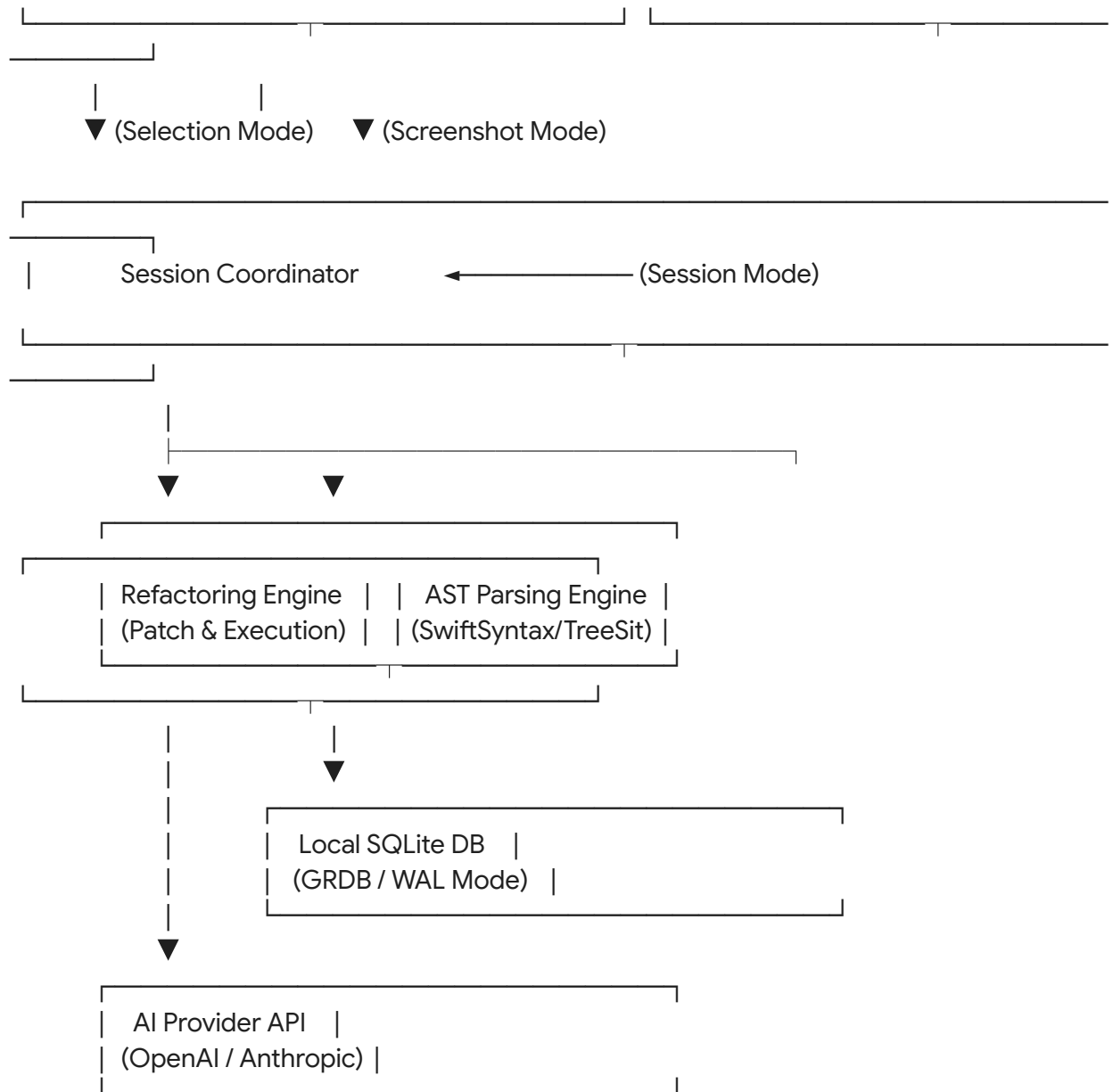
This document specifies the production-ready technical architecture for Decode, a local, sandboxed macOS developer companion. Decode is designed to provide real-time code understanding, structural dependency analysis, automated local refactoring, and on-demand code explanations from any application on macOS.

The system supports three distinct operating modes:

- 1. **Session Mode:** Operates strictly within a stateful Single Session / Single File context scope. It translates raw code changes from a tracked file on disk into a semantic network of code entities, logical notes, structural relationships, and automated refactoring actions.
- 2. **Selection Mode (New):** A stateless mode that captures highlighted code from any macOS application via a global hotkey, generating immediate semantic explanations without creating a database session.
- 3. **Screenshot Mode (New):** A stateless fallback mode that launches an interactive screen-crop overlay when no text is selected, extracting code from the captured region via a native OCR pipeline to generate immediate explanations.

Unified System Context Diagram

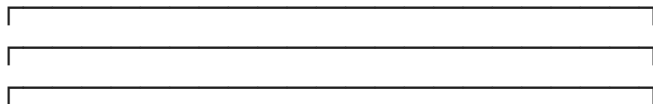


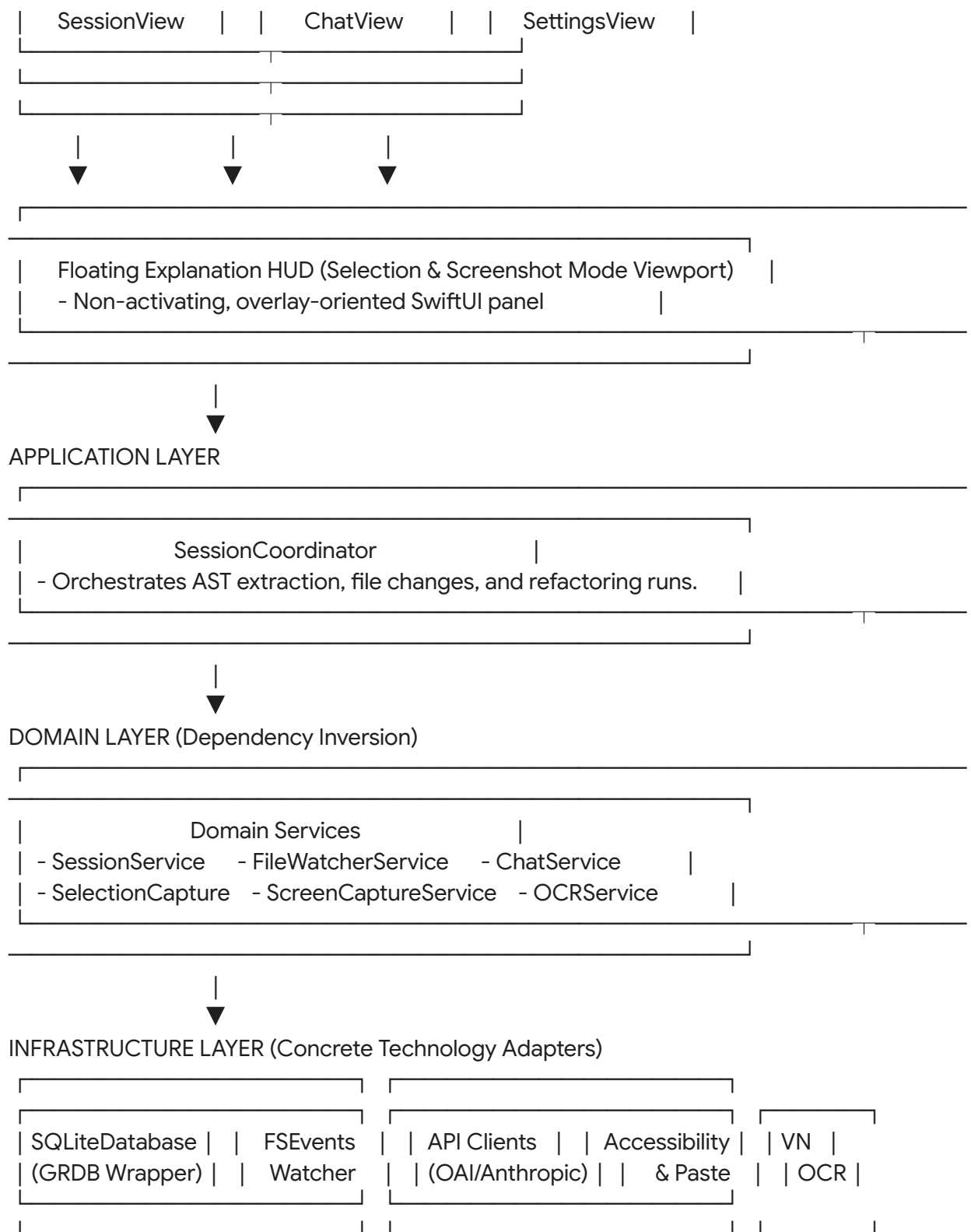


2. Component Architecture Diagram

The system utilizes a strictly decoupled, unidirectional architectural design consisting of four layers. Communication across these layers is mediated via protocols to ensure testability and strict separation of concerns.

PRESENTATION LAYER





3. Layered Architectural Principles

Separation of Concerns

Components are strictly separated into horizontal slices. Elements of any given layer can only interact with entities located in their own tier or tiers below. Dependencies flow downward: Presentation -> Application -> Domain -> Infrastructure.

Dependency Inversion Protocol

The Domain layer defines abstract, technology-agnostic protocols representing data and integration boundaries. The Infrastructure layer implements these interfaces. If the target SQLite wrapper (GRDB), the native OCR engine, or the underlying LLM networking details change, the Domain and Presentation layers remain unaffected.

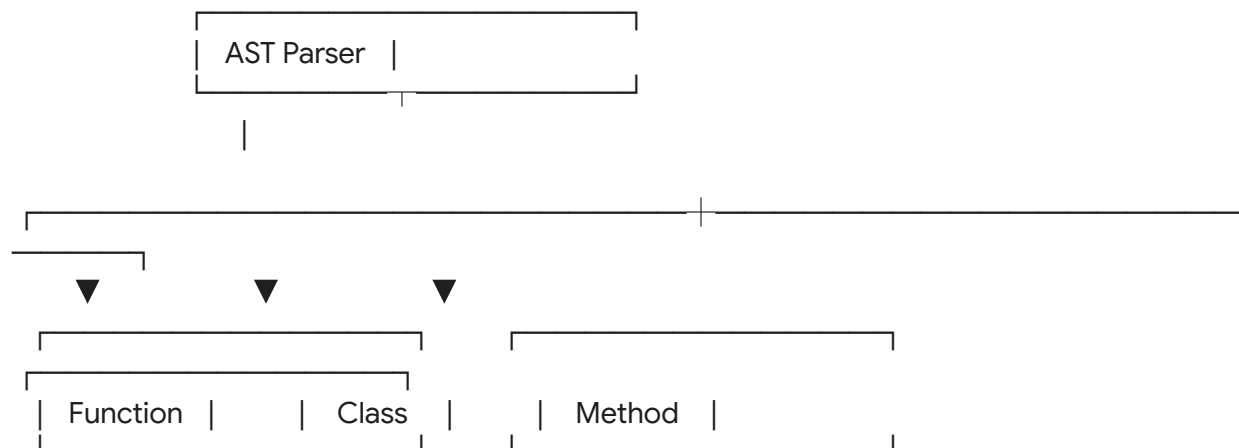
Concurrency & Data Isolation

To ensure the UI remains responsive, all compute-heavy or blocked I/O tasks are bound to background Execution Contexts.

- **Database Concurrency:** SQLite is configured to utilize Write-Ahead Logging (WAL) to support non-blocking concurrent reads during active background writes.
- **Thread Isolation:** Background interactions (AST analysis, hashing, OCR text extraction, and LLM message serialization) run isolated on dedicated background Actors, preserving the MainActor for interface rendering.

4. AST & Entity-Driven Subsystem Architecture (Session Mode)

The MVP replaces raw file summaries with an AST-driven indexing engine. Instead of treating the file as an opaque block of text, the system decomposes it into distinct semantic units known as **Code Entities**.



4.1 AST Parser Subsystem Responsibilities

1. **Entity Extraction:** Detects and parses semantic blocks using compiler-validated syntax trees:
 - o *Procedural Code:* Functions, Methods.
 - o *Object-Oriented Blocks:* Classes, Structs, Protocols.
 - o *Component-Driven Frameworks:* Components, Custom Hooks (e.g., React/SwiftUI structures).
2. **Dependency Tree Synthesis:** Extracts logical interactions and dependencies between entities inside the file, constructing a lightweight structural call-graph.
3. **Change Detection Mapping:** Identifies structural changes (creates, updates, deletes) on a per-entity basis during saving.

Note: The AST Parser functions purely as an indexing and analysis engine. Robust syntax trees derived from libraries like SwiftSyntax, Tree-sitter, or language-specific parsers form the core parsing foundation. Regular expressions (Regex) are restricted strictly to basic post-processing string sanitization. The physical source file on disk remains the absolute source of truth.

4.2 Semantic Entity Model Design

Entities are identified exclusively by their semantic structural footprint rather than unstable physical line numbers. This guarantees that edits above an entity do not invalidate its identity.

Entity
- entityId: UUID (Primary Key) - stableEntityId: String (Durable symbol identifier) - entityType: Enum (function class method hook) - name: String (Symbolic identifier) - summaryText: String (Purpose, responsibilities) - hash: String (SHA-256 of the entity's body) - lastUpdated: Date

4.3 Structural Dependency Graph Subsystem

To facilitate deep contextual assembly for both explanation and refactoring, the system

constructs a lightweight structural dependency graph. It represents call and composition relationships inside the code:

$$\text{login()} \longrightarrow \text{validatePassword()} \longrightarrow \text{getUser()}$$

This graph is structurally tracked in database junction tables and is devoid of vector database dependencies or semantic approximations. It maps call-graph lineages (i.e., Entity A references Entity B).

4.4 Notes Subsystem (1 Note = 1 Code Entity)

Every discovered Entity is linked directly to a Note. Each Note contains:

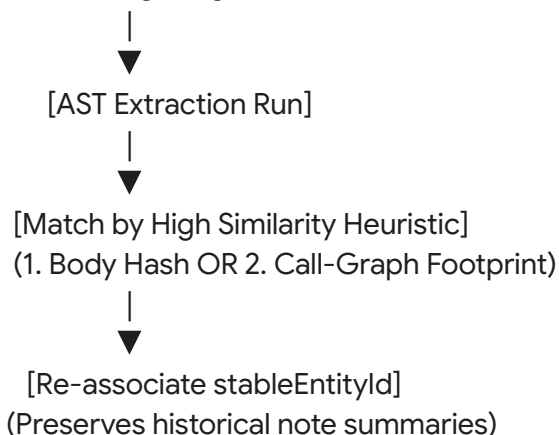
1. **Purpose:** High-level narrative on what the code block accomplishes.
2. **Responsibilities:** Bullets describing the operations of the entity.
3. **Relationships:** Structural references to calling and called dependencies.

The high-level File Summary is decoupled from Entity Notes. It describes only the overall file purpose, major architectural constructs, and high-level responsibilities of the file as an integrated module.

4.5 Entity Identity Strategy

To prevent entity identity degradation during simple symbol renames, the system implements a `stableEntityId` mapping layer.

[Code Change: `login()` -> `authenticateUser()`]



`stableEntityId` Requirements

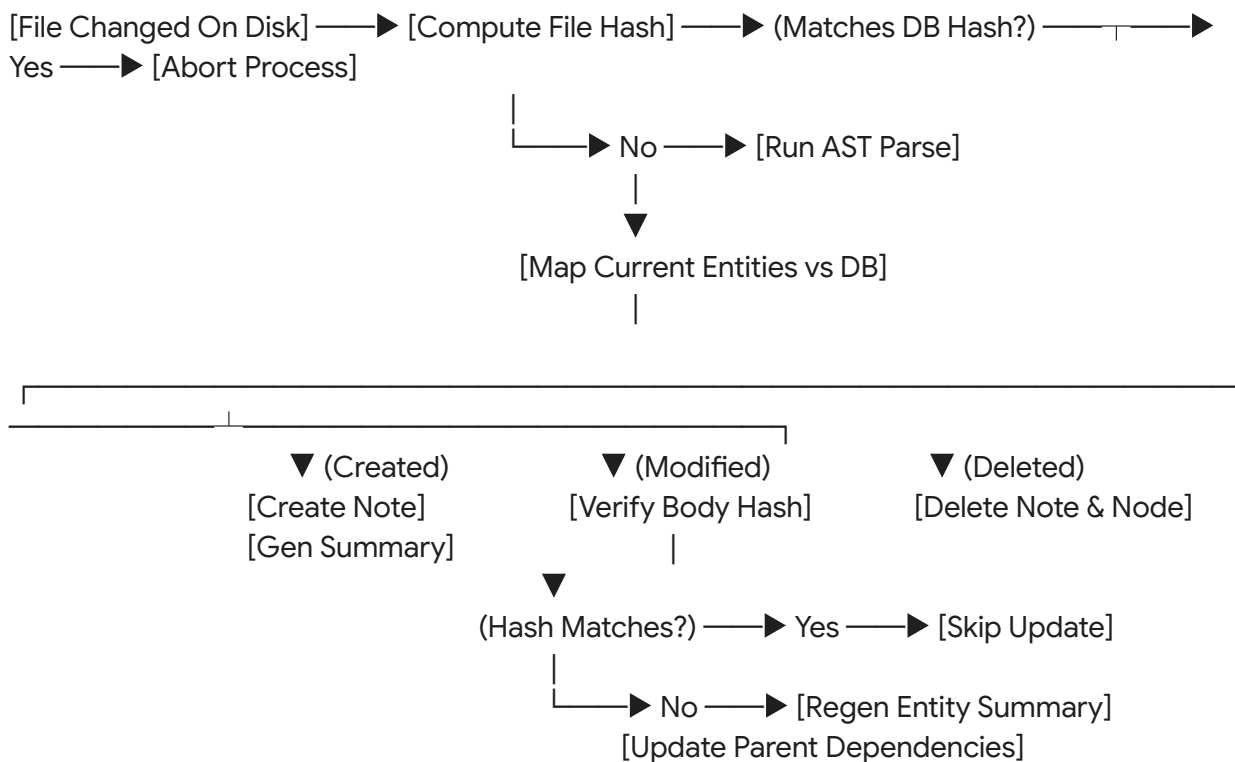
1. **Durable Lifetime:** Generated when an entity is first discovered and written to the database. It remains constant across file saves and automated refactoring runs.
2. **Rename Preservation Heuristics:** If an entity name changes but its body hash,

parameter signature, or local call-graph dependencies exhibit high similarity (defined by a matching similarity threshold $S \geq 0.85$ over an AST subtree structure), the engine reconciles it as a rename of the existing record rather than a delete-and-recreate cycle.

3. **Note Association Preservation:** By tracking the entity history through the stableEntityId instead of the raw symbol name, user summaries, notes, and historical context are maintained.
4. **Future Heuristic Extensions:** Scale to utilize Tree-sitter node structural hashing or abstract syntax subtree comparison to ensure identity durability during combined renames and minor edits.

5. File Change Workflow & Selective Summary Update (Session Mode)

This design replaces the computationally expensive practice of regenerating the entire file context when changes occur with an incremental, selective pipeline.



5.1 Selective Summary Regeneration Advantages

By focusing AI operations exclusively on modified entities, the companion optimizes runtime resource allocation:

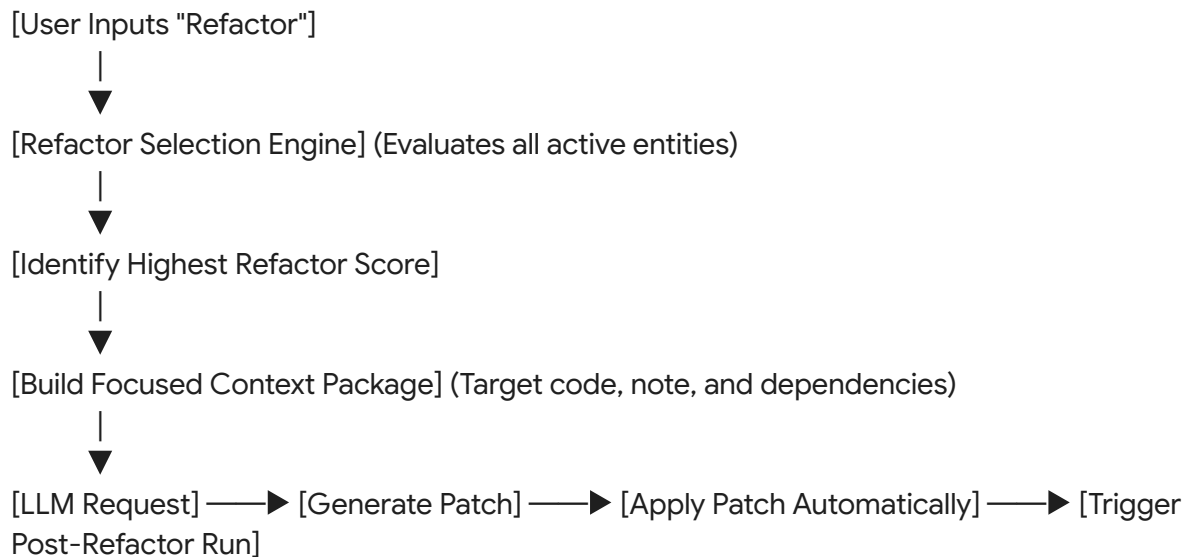
- **API Cost Reduction:** Processing single functions (typically 100–400 tokens) instead of

resubmitting whole files (10k–50k tokens) reduces prompt token overhead by 80% to 95%.

- **Latency Reduction:** LLM response times scale with generated sequence length. Generating structural summaries for individual entities reduces processing time from minutes to seconds.
- **Context Preservation:** Selective tracking preserves older, manual notes and unchanged entity histories, avoiding degradation from repeated regenerations of unchanged code.

6. Refactoring Engine Architecture (Session Mode)

The Refactoring Engine identifies, isolates, refactors, and reconciles changes to targeted logical blocks without manual selection or complex IDE highlighting wrappers.



6.1 Refactor Selection Engine & Scoring Model

When a user requests a refactor without specifying a target, the Refactor Selection Engine analyzes the active file's AST entities. It assigns a Refactor Score to each entity to prioritize highly complex, deeply nested, or over-dependent areas.

$$\text{Score} = \text{function_length} + (\text{nesting_depth} \times 3) + (\text{dependency_count} \times 2)$$

Precise Metric Specifications

- **function_length** : Evaluated as the exact AST statement count within the body of the entity. Statement count is strictly preferred over physical line counts because it is formatting-agnostic (whitespace, line wraps, brace placement do not alter the complexity score) and cleanly excludes inline comments and blank lines.

- **nesting_depth** : The maximum nested block scope depth within the target entity (e.g., conditional checks, loops, pattern matches).
- **dependency_count** : The count of unique outgoing call dependencies (fan-out) linked to this entity in the local dependency graph.

Metric Calculation Examples

// Example Entity A: Low Complexity

```
func logMessage(msg: String) {
    print(msg) // statement_count = 1, nesting_depth = 0, dependency = 0
}
```

// Score for Entity A: $1 + (0 * 3) + (0 * 2) = 1$

// Example Entity B: High Complexity

```
func processPayment(order: Order) throws {
    if order.isValid { // nesting_depth = 1
        for item in order.items { // nesting_depth = 2
            if item.price > 0 { // nesting_depth = 3
                try gateway.charge(item) // dependency_count = 1 (gateway.charge)
            } else {
                throw PaymentError.invalidPrice
            }
        }
    }
    db.save(order) // dependency_count = 2 (db.save)
}
```

// AST Statement Count:

// 1 (if), 2 (for), 3 (if nested), 4 (charge), 5 (throw), 6 (save)

// Score for Entity B: $6 + (3 * 3) + (2 * 2) = 6 + 9 + 4 = 19$ (High Priority Target)

6.2 Refactor Context Building Strategy

Once the engine selects a target entity, it compiles a compact context package. It does not send the entire file context.

Refactoring Prompt Context	
1. High-Level File Summary (Aesthetic context of the overall file)	
2. Target Entity Source Code (The exact code block to be rewritten)	

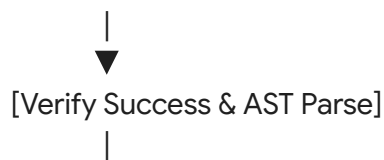
- | 3. Target Entity Note Summary (Purpose, constraints, and features) |
 - | 4. Structural Dependency Summaries (Abstract signatures of called APIs) |
 - | 5. Core System Prompt (Refactoring paradigms, execution constraints) |
-

By isolating the context window, the system minimizes attention dilution, reduces processing costs, and increases the precision of the output patch.

6.3 Refactor Execution & Automated Patch Application

1. **Request Dispatch:** The built context is transmitted to the AI Provider, requesting a clean, structured patch file or replacement representation.
2. **Automated Application:** The engine automatically applies the patch directly to the file on disk. To maintain simplicity, the MVP implements no approval step, no diff preview, and no undo UI.
3. **Patch Application Safety Guard (Crash Recovery):** Before writing the modified patch directly to the file on disk, the system executes an atomic memory backup transaction.

[Retrieve Current File Content] —► [Cache Temporarily in Memory] —► [Apply Generated Patch]



▼ (Successful)
[Persist to Disk]
[Rebuild DB Index]

▼ (Failure)
[Restore Cached Data]
[Raise System Error]

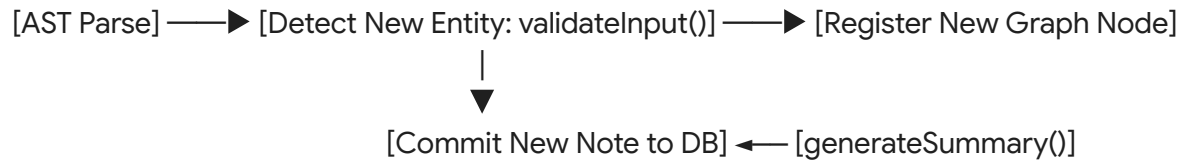
Note: This process does not provide interactive user undos, manual visual diff comparisons, or approval popups. It is strictly a low-level crash-recovery protection mechanism to prevent filesystem truncation or corrupted half-writes if the LLM output patch generation or filesystem stream fails mid-process.

4. **AST Reconciliation:** Once the changes are safely committed to disk, the FileWatcherService detects the update, prompts the AST Parser to re-run, discovers any structural shifts, and updates the UI accordingly.

6.4 Post-Refactor Entity Discovery Workflow

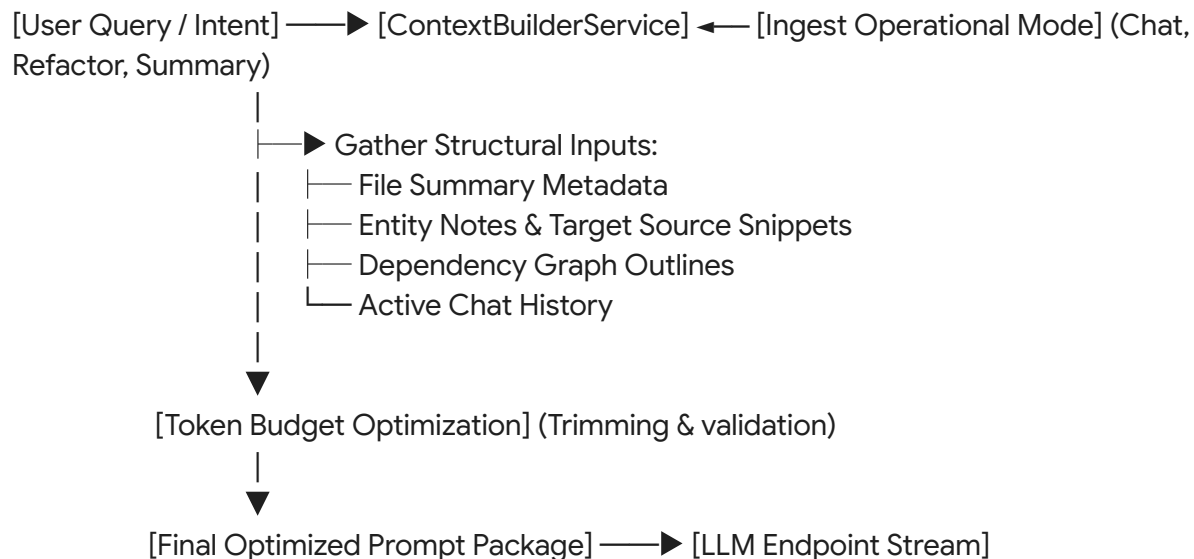
If a refactoring operation introduces a new helper function (e.g., extracting logic from login())

into a new helper called `validateInput()`, the system handles it dynamically:



7. Context Builder Architecture (Session Mode)

The Context Builder is a dedicated, centralized subsystem. It compiles raw database states, local prompt structures, dynamic chat threads, and operational constraints into single, highly optimized instruction packages.



7.1 Responsibilities

1. **Chat Context Assembly:** Evaluates user questions, determines relevant code regions, and extracts the minimal required set of surrounding notes, summaries, and chat history.
2. **Refactor Context Compilation:** Packages target entity source code, its notes, dependent functions' signatures, and high-level file targets.
3. **Summary Context Preparation:** Builds standard prompts with structural outlines for generating summaries of newly created or altered entities.
4. **Token Budget Optimization:** Performs pre-flight token estimates. If estimated usage exceeds target model boundaries, it implements graceful degradation (e.g., trimming old chat history, excluding distant dependencies) to stay within constraints.

7.2 Centralization Justification

Spreading prompt-building logic across multiple disjointed view models or domain services

creates fragile dependency loops. Centralization in the ContextBuilderService ensures:

- **Uniform Prompt Formatting:** Structural formatting rules and output directives are enforced at a single gateway.
- **Consolidated Token Auditing:** A single point checks constraints and sizes, preventing unexpected overflows.
- **Separation of Concerns:** Business execution logic is kept clean of template formatting, model directives, and LLM-specific structures.

8. Code Understanding & Chat Context Workflow (Session Mode)

Code-understanding conversations are powered by a context pipeline designed to minimize token usage and model confusion.

8.1 Resolution Workflow

[User Submits Question]



[Analyze Question Keywords] (Identify referenced entities & call routes)



[Retrieve Relevant Entity Notes] (Exclude non-referenced code bodies from DB)



[Retrieve Dependency Summaries] (Assemble abstract calling signatures from DB)



[Retrieve Global File Summary] —> [Build Context Prompt Package] —> [Send Stream to Provider]

8.2 Token Optimization and Accuracy Drivers

- **Targeted Context Injection:** Rather than sending the entire source file for every turn (which creates high latency and can lead to model distraction), the system analyzes the user query, identifies referenced entities, and loads only those entity notes and code bodies.
- **Token Mitigation via Notes:** Entity notes provide semantic summaries of non-target code segments, preserving the LLM's system awareness while omitting physical code implementation details.
- **Structural Call Accuracy:** Dependency summaries define what external methods accept and return without cluttering the context window with their underlying function

implementations.

8.3 Context Selection Example

Scenario: A user queries: *"Why does processPayment() fail with invalidPrice errors?"*

Context Package Assembled by ContextBuilder	
1. Core Target Code (Actual body of processPayment())	
2. Dependencies (Summary notes of database.save and gateway.charge)	
3. Global File Context (Overview that this file handles transactions)	
4. Excluded Blocks (AST bodies for validateAddress, processReceipt)	

This reduces the prompt payload from 12,000 tokens of raw source code to a highly targeted 1,200-token context package, optimizing latency and focus.

9. Session Runtime Model (Session Mode)

The Session object is the central runtime entity. It manages structural code graphs, transactional states, and active memory scopes.

Session (Root)	
- sessionId: UUID	
- fileMetadata: FileMetadata (File path, modification state, hashes)	
- fileSummary: FileSummary (Overall module description)	
- entities: EntityCollection (AST discovered entities via stableId)	
- notes: NotesCollection (Linked documentation & metadata summaries)	
- dependencyGraph: DependencyGraph (Inter-entity calling structures)	
- chatHistory: ChatHistory (History of active conversation turns)	

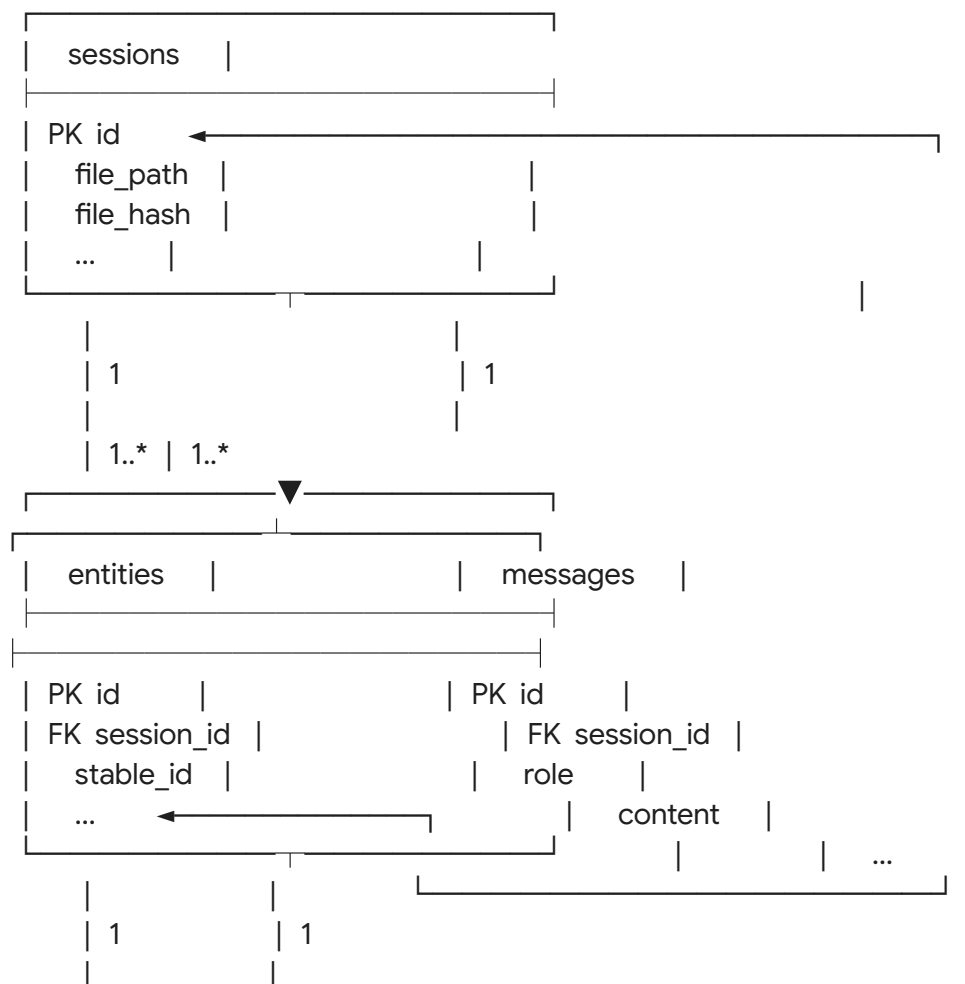
Component Ownership & Lifecycles

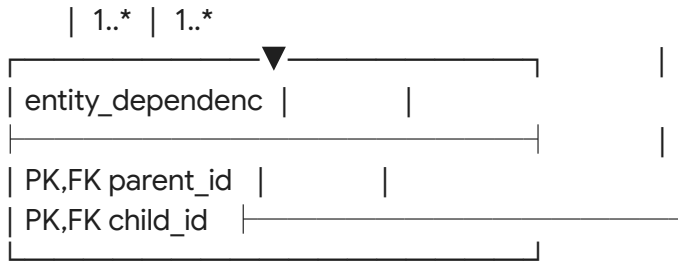
- **Session (Aggregate Root):** Dictates the structural bounds of the active sandbox window. It owns all metadata, structural entities, documentation notes, and chat threads. Removing a Session from memory cleanly unloads all nested collections.
- **Entity Collection:** Tracks the current structural components extracted by the AST parser. It handles re-indexing runs and maintains association maps with the DependencyGraph.
- **Notes Collection:** Linked directly to the EntityCollection via the stableEntityId. Notes store historical summaries even if the underlying code is modified or moved.
- **Dependency Graph:** Defines the structural connections between entities. It updates in step with AST indexing runs.
- **Chat History:** An independent, append-only collection that tracks conversational messages and streaming updates.

10. Data Model & Database Design (Session Mode)

The system's persistent state is managed by a structured SQLite instance configured for rapid retrieval and referential integrity.

10.1 Relational Schema Diagram





10.2 Database DDL (Schema Definition)

-- Enable foreign keys support

PRAGMA foreign_keys = ON;

-- Sessions Table (Host Container representation)

```

CREATE TABLE sessions (
    id TEXT PRIMARY KEY NOT NULL,
    created_at TEXT NOT NULL,
    updated_at TEXT NOT NULL,
    bookmark_data BLOB NOT NULL,
    file_path TEXT NOT NULL,
    file_name TEXT NOT NULL,
    file_size INTEGER NOT NULL,
    file_mod_at TEXT NOT NULL,
    file_hash TEXT NOT NULL,
    summary_text TEXT NOT NULL,
    is_corrupted INTEGER NOT NULL DEFAULT 0
) WITHOUT ROWID;
  
```

-- Entities Table (Contains the Notes structural data mappings)

```

CREATE TABLE entities (
    id TEXT PRIMARY KEY NOT NULL,
    session_id TEXT NOT NULL,
    stable_id TEXT NOT NULL,
    entity_type TEXT NOT NULL CHECK(entity_type IN ('function', 'class', 'method', 'component',
'hook')),
    name TEXT NOT NULL,
    summary_text TEXT NOT NULL,
    hash TEXT NOT NULL,
    last_updated TEXT NOT NULL,
    FOREIGN KEY (session_id) REFERENCES sessions(id) ON DELETE CASCADE
) WITHOUT ROWID;
  
```

```
-- Entity Dependencies Junction Table (Lightweight Dependency Graph)
CREATE TABLE entity_dependencies (
  parent_id TEXT NOT NULL,
  child_id TEXT NOT NULL,
  PRIMARY KEY (parent_id, child_id),
  FOREIGN KEY (parent_id) REFERENCES entities(id) ON DELETE CASCADE,
  FOREIGN KEY (child_id) REFERENCES entities(id) ON DELETE CASCADE
) WITHOUT ROWID;

-- Messages Table (Transactional Chat History)
CREATE TABLE messages (
  id TEXT PRIMARY KEY NOT NULL,
  session_id TEXT NOT NULL,
  role TEXT NOT NULL CHECK(role IN ('user', 'assistant', 'system')),
  content TEXT NOT NULL,
  created_at TEXT NOT NULL,
  FOREIGN KEY (session_id) REFERENCES sessions(id) ON DELETE CASCADE
) WITHOUT ROWID;
```

10.3 Performance Indexes & SQLite Optimizations

- **WITHOUT ROWID Optimization:** Applied across all core tables. Because the keys are non-integer UUID strings (TEXT), this eliminates the overhead of a secondary B-Tree lookup for row identifiers.
- **Cascade Delete Constraints:** Configured on downstream tables (entities, entity_dependencies, messages). Purging a primary session cleanly removes its associated sub-trees in a single transactional step.
- **ISO8601 Temporal Ingestion:** Timestamps are recorded as string formats (YYYY-MM-DD HH:MM:SS) to support reliable lexicographical sorting.
- **Performance Indexes:**

```
CREATE INDEX idx_messages_session ON messages(session_id);
CREATE INDEX idx_messages_session_time ON messages(session_id, created_at);
CREATE INDEX idx_entities_session ON entities(session_id);
CREATE INDEX idx_entities_stable ON entities(stable_id);
```

11. MVP Scope Exclusions (Session Mode)

To maintain stability and focus on the core product vision, the MVP does not support:

- **Multi-File Sessions / Workspace Ingestion:** No logical traversal outside the single user-targeted code file.
- **Recursive Directory Parsing:** No workspace-level AST exploration or parent directory

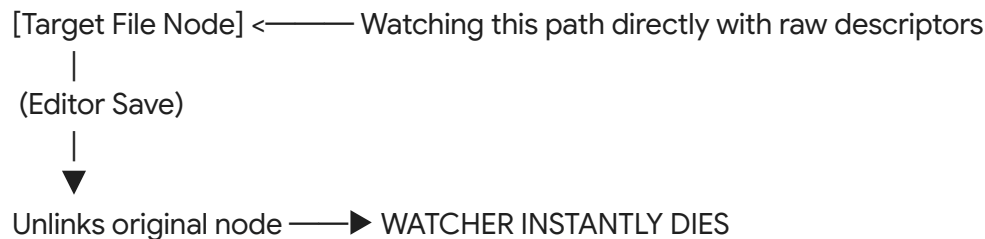
structural indexing.

- **Vector Databases / Local Embeddings / RAG Pipelines:** No semantic parsing engines, vector indexes, or token embedding conversions.
- **Autonomous Agent Iterative Execution Loops:** The system does not write code, compile, evaluate terminal feedback, and self-correct in iterative loops.
- **Auto-Refactor-On-Save:** Automatic code modifications are never triggered on save. Changes are generated only on manual user request.

12. File Monitoring, File Recovery, & System Resiliency (Session Mode)

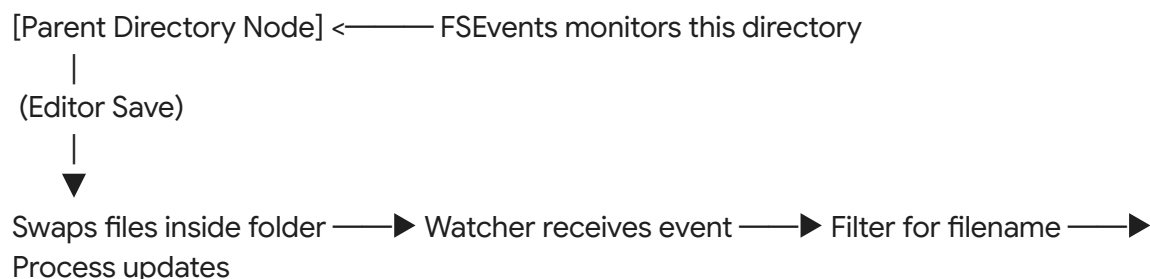
12.1 The Atomic Save Challenge

Most modern IDEs and text editors do not modify target files in place. Instead, they write to a temporary file, delete the original path, and rename the temporary file to match the target.



Directory-Level Monitoring Solution

To persist across atomic saves, the FSEventsWatcher monitors the parent directory of the target file, applying a path filter to match the target file's name.



12.2 Change Detection & Hashing Pipeline

When a change notification is received:

1. **Metadata Inspection:** Query file attributes (size and modification date). If they match existing database values, discard the event immediately.
2. **Streaming Cryptographic Hash:** If the metadata differs, read the file in small chunks

using a streaming SHA-256 process to minimize memory overhead. Do not load the entire file into memory at once.

3. **State Evaluation:** Compare the new hash against sessions.file_hash. If it matches, the file was saved without edits (e.g., a file touch). If it differs, mark the session as out of sync (is_dirty).

```
import CryptoKit
import Foundation
```

```
struct SafeHashUtility {
    static func computeSHA256(for url: URL) throws -> String {
        let handle = try FileHandle(forReadingFrom: url)
        defer { try? handle.close() }
        var hasher = SHA256()
        while let chunk = try handle.read(upToCount: 65536), !chunk.isEmpty {
            hasher.update(data: chunk)
        }
        return hasher.finalize().map { String(format: "%02hhx", $0) }.joined()
    }
}
```

12.3 System Error Handling Matrix

Failure Event	Root Cause	System Diagnostic State	User Recovery Strategy
File Deleted	User removed file from outside the app.	System throws NSFileReadNoSuchFileError.	Mark session as "Offline". Keep the last saved summary active, disable chat inputs, and display a warning banner.
File Moved	Target file renamed or dragged to another folder.	Inode path resolution failure.	Resolve via Security-Scoped Bookmark. If the bookmark resolves, silently update the DB path. If it fails, transition to the "File Deleted" recovery workflow.

File Inaccessible	Access token expired or sandboxed directory permission revoked.	OS sandbox access violation.	Show an explicit prompt with an action to re-select the file using <code>NSOpenPanel</code> to grant new access permissions.
AI Provider Offline	Network down, timeout, or invalid API key.	HTTP status 401, 503, or connection timeout.	Catch the exception, display an inline warning under the input box, and preserve the user's typed message in the input field.
Database Failure	Drive out of space or SQLite database lock.	SQLite I/O error or write failure.	Transition to an in-memory fallback state. Alert the user to free up disk space and prevent further database writes.
Corrupted Session	Malformed SQLite records or parsing failures.	Parsing failure during fetch operations.	Set <code>is_corrupted = 1</code> . Render a diagnostic UI offering three recovery paths: 1. Rebuild metadata from disk, 2. Re-import target file, or 3. Clean manual delete.

13. AI Provider Integration & Network Resiliency

All LLM communication is decoupled behind the `AIProvider` protocol.

```
protocol AIProvider: Sendable {
    func generateSummary(fileContent: String, systemPrompt: String) async throws -> String
```

```
func streamChat(messages: [Message], systemPrompt: String) async throws ->
AsyncThrowingStream<String, Error>
}
```

Resiliency Mechanics

- **Exponential Backoff with Jitter:** For transient network failures, retry up to 5 times. The sleep interval is computed as:

$$T_{\text{sleep}} = 2^{\text{attempt}} + \text{random_jitter}$$

- **Rate Limit (HTTP 429) Management:** If an HTTP 429 status code is received, pause outgoing tasks, extract the Retry-After header if present, and surface a dedicated rate-limit notice in the UI.

14. Security Design & Sandbox Bypass

Security-Scoped Bookmarks

To bypass macOS sandboxing restrictions across application restarts, file paths are persisted using security-scoped bookmarks:

```
// 1. Create a secure bookmark
let bookmarkData = try url.bookmarkData(
    options: .withSecurityScope,
    includingResourceValuesForKeys: nil,
    relativeTo: nil
)

// 2. Resolve secure bookmark upon subsequent app launches
var isStale = false
let resolvedURL = try URL(
    resolvingBookmarkData: bookmarkData,
    options: .withSecurityScope,
    relativeTo: nil,
    bookmarkDataIsStale: &isStale
)

let accessGranted = resolvedURL.startAccessingSecurityScopedResource()
defer { resolvedURL.stopAccessingSecurityScopedResource() }
```

Local Secret Storage

API keys must never be stored in plaintext property lists, JSON configurations, or standard UserDefaults containers.

- **Mechanism:** Use the macOS Keychain Services API directly.
- **Security Parameter:** Set records with `kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly` to ensure they are encrypted at rest and excluded from device backup exports.

15. Selection Mode Architecture (Mode 2)

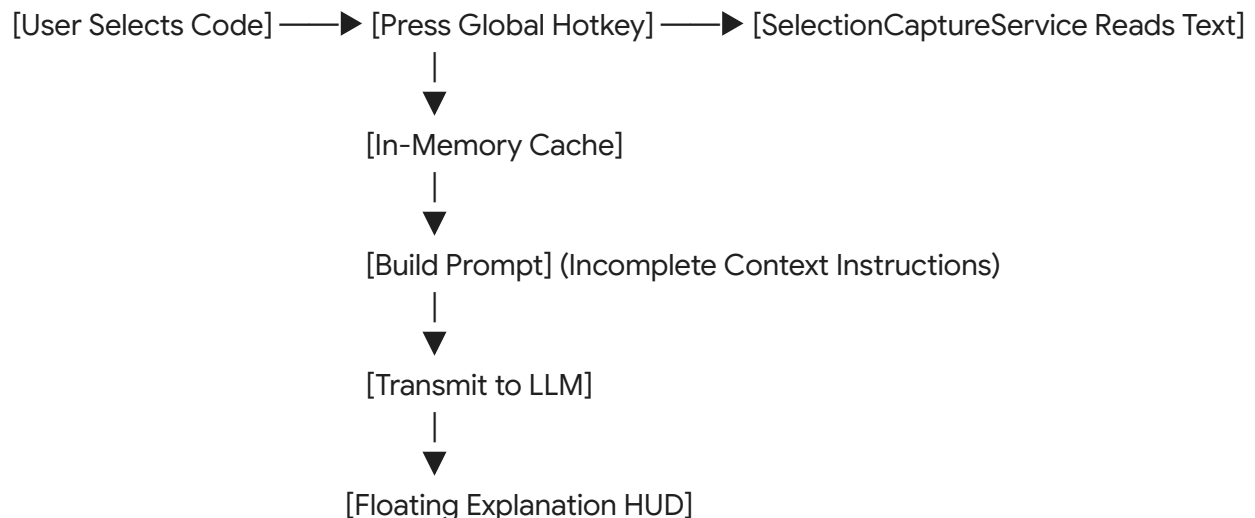
15.1 Purpose & Target Scope

Selection Mode allows users to instantly explain code highlighted anywhere on macOS without creating a session, writing to the database, or setting up a persistent file watcher.

Supported Sources

- Native PDF readers (e.g., Preview, Adobe Acrobat)
- Web browsers (e.g., Safari, Chrome, Firefox)
- Web repositories and forums (e.g., GitHub, Stack Overflow)
- Documentation portals (e.g., Apple Developer Docs, MDN)
- Knowledge management and notes tools (e.g., Notion, Apple Notes)
- IDEs and external editors (e.g., Xcode, VS Code, Sublime Text)
- Any active macOS application exposing selectable text elements.

15.2 Functional Workflow



15.3 Core Characteristics (Stateless Boundary)

To maintain strict architectural isolation, Selection Mode is strictly **stateless**:

- **No Database Writes:** The extracted code, user query, and assistant output bypass the SQLite engine entirely.
- **No File System Monitoring:** No active FSEvents watches or file paths are resolved.
- **No AST Parsing / Code Graphs:** The parsed block is treated as an isolated, memory-resident text buffer. No structural dependency modeling or symbol indexing is performed.
- **Zero Persistence:** All states live purely in-memory. Once the floating explanation HUD is dismissed, the associated memory buffer is deallocated.

15.4 Incomplete Context Strategy

Because Selection Mode captures arbitrary, isolated text fragments, the underlying compiler/symbol framework is missing. This introduces a high risk of LLM hallucinations regarding variable types, missing imports, and unresolved dependencies.

The System Prompt for Selection Mode must strictly enforce the **Incomplete Context Strategy**, requiring the model to partition its analysis into three highly explicit boundaries:

1. **High-Confidence Findings:** Explanations of logic, syntax, and patterns that are fully self-contained and mathematically/logically verifiable from the extracted snippet alone.
2. **Probable Interpretations:** Strongly inferred context, typical framework patterns, or suspected library usages (e.g., assuming a `$`-prefixed variable represents a SwiftUI binding or a React state setter). These must be framed explicitly as high-probability inferences rather than absolute facts.
3. **Unknown / Missing Context:** A dedicated section listing critical dependencies, uninitialized variables, structural imports, or external configurations required to run or fully validate the code.

By separating certainty from logical inference, this system maintains absolute transparency, reducing the cognitive load on the developer to verify the explanation's validity.

16. Screenshot Mode Architecture (Mode 3)

16.1 Purpose

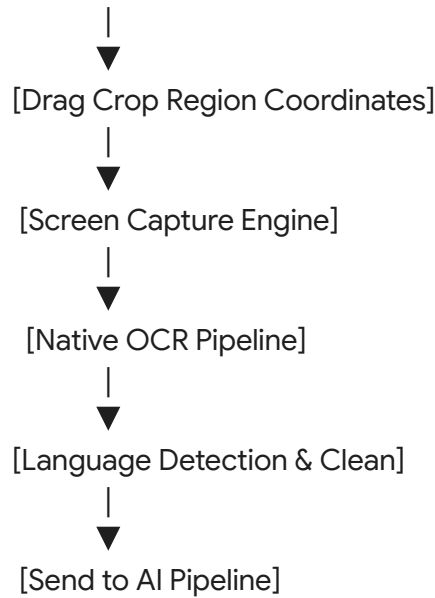
Screenshot Mode provides fallback support to extract and explain code from sources where visual rendering inhibits text selection.

Targets

- Images, diagrams, and static presentations (e.g., Keynote, slides)
- Scanned or rasterized non-searchable PDF documents
- Video-based instructional content (e.g., YouTube tutorial streams, lecture recordings)
- Social media feeds and workspace screenshots (e.g., Slack images, Twitter code shares)

16.2 Functional Workflow

[Global Hotkey Pressed] —▶ (Active Selection?) —▶ No —▶ [Launch Overlay Window]

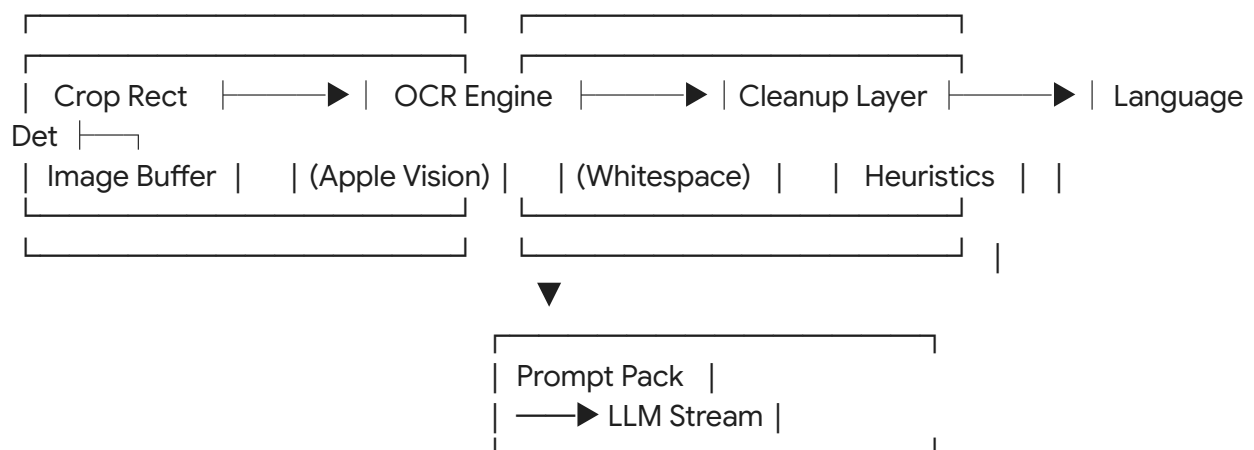


16.3 Operational Characteristics

- **Statelessness:** Operates strictly on a volatile, memory-resident frame buffer. No temporary PNGs are persisted to the filesystem, and no database writes are generated.
- **Interactive HUD Execution:** Leverages custom-positioned UI elements to prevent focus stealing from the source application.

17. OCR Pipeline Architecture

To extract structural syntax from visual capture coordinates, Screenshot Mode utilizes a specialized, native pipeline optimized for source code layouts.



17.1 Step 1: Image Capture

The visual region bound by the user's drag coordinates is captured utilizing a low-overhead screen-capture engine. The raw frame buffer is passed directly to the OCR coordinator as a memory-resident CGImage slice.

17.2 Step 2: Native OCR Processing (OCRService)

The system invokes the native Apple **Vision Framework** (VNRecognizeTextRequest). The service is configured for maximum architectural fidelity:

- **Recognition Level:** Set to .accurate (utilizes deep convolutional neural networks rather than the fast, low-power heuristic model).
- **Language Features:** .usesLanguageCorrection is set to false. *Architectural Justification:* Standard language correction engines corrupt coding syntax (e.g., altering programmatic variables like `func init()` into plain English words like "function initial").

17.3 Step 3: OCR Error Cleanup & Formatting Reconstruction

The output raw string buffer from the Vision API is passed through a deterministic post-processing layer to resolve common scanning errors:

- **Brace and Symbol Normalization:** Identifies and corrects common character misidentifications within structural symbols (e.g., reconstructing a mis-scanned 1 or l inside logical evaluations, restoring [and] brackets, or correcting the character mapping of relational operators like < and >).
- **Indentation Alignment Heuristic:** Analyzes leading whitespace of raw text coordinates to reconstruct correct scope indentation depths, preventing flat-left-aligned code blocks from being passed to the prompt compiler.
- **Line Splice Verification:** Re-evaluates lines that have been artificially split by visual line-breaks, merging them safely back into syntactically valid continuous statements.

17.4 Step 4: Language Detection

The extracted code is run through a dual detection heuristic:

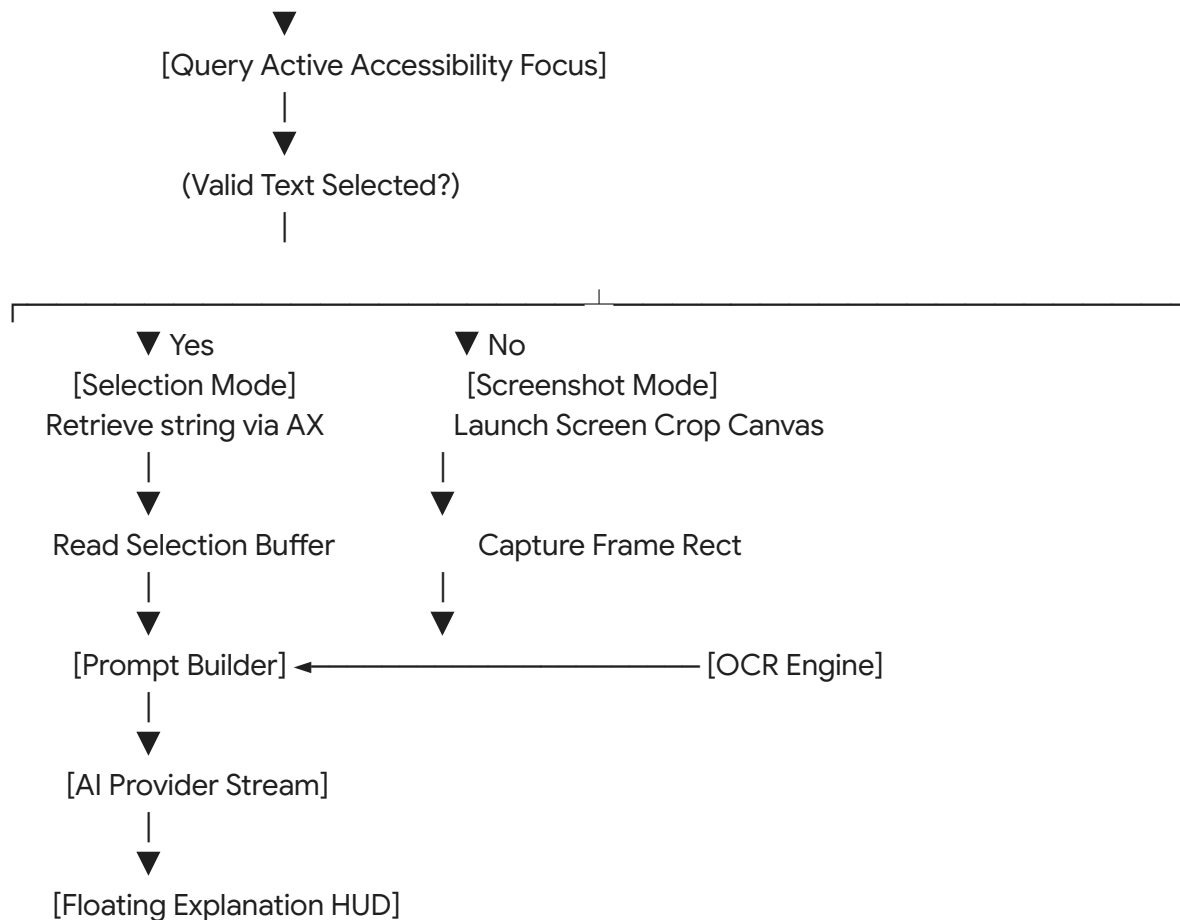
1. **Heuristic Keyword Profiling:** Scans the text buffer for language-specific syntactic anchors (e.g., `import React -> TypeScript/JavaScript`, `func and struct -> Swift`, `#include <iostream> -> C++`).
2. **Natural Language Identifier Fallback:** Invokes Apple's native NLLanguageRecognizer to classify comments and identifier namespaces, validating the targeted programming language environment.

18. Unified Hotkey & Input Capture Strategy

To minimize UI footprint and provide a friction-free macOS user experience, Decode routes input dynamically via a **single, unified global hotkey shortcut** (Default: `⌘ Space`).

[User Presses Hotkey]

|



Routing Logic

1. **Trigger Event:** The user registers a system-wide hotkey event using the `GlobalHotkeyManager`.
2. **Accessibility Evaluation:** The `SelectionCaptureService` queries the active window using native Accessibility APIs (`AXUIElementCopyAttributeValue`).
3. **Failable Pipeline Branching:**
 - **Branch A (Selection Mode):** If a text-selection attribute is successfully resolved and returned within a strict 150ms timeout gate, the captured text buffer is immediate-forwarded to the Prompt Builder.
 - **Branch B (Screenshot Mode Fallback):** If the accessibility query returns null, throws a permission-denied error, or fails to resolve within the timeout gate, the system drops through to Screenshot Mode, spawning the transparent crosshair cropping window overlay.

19. Additive macOS Subsystems

To preserve the clean decoupling of the system, all native integrations are engineered as

additive infrastructure adapters. **No modifications are made to Session Mode services.**

GlobalHotkeyManager

- Low-level listener utilizing Carbon's RegisterEventHotKey framework.
- Binds a user-defined key combination globally.
- Forwards activation events directly to the Selection Capture router.

SelectionCaptureService

- Inspects system-wide focus using Cocoa Accessibility APIs.
- Fallback Hook: If AX focus fails to return selected text, executes a synthesized 'Cmd+C' keystroke sequence, reads the NSPasteboard buffer, and safely restores prior clipboard state.

ScreenCaptureService

- Generates a non-activating, overlay-level window.
- Implements mouse-tracking drag listeners to draw crop boundaries.
- Translates screen coordinates into physical pixel rect bounding box.
- Captures visual bounds using CGDisplayCreateImage framework.

OCRService

- Integrates directly with the Apple Vision (VNText) framework.

- Performs non-correction text recognition on high-resolution buffers. |
- Applies line formatting, bracket normalization, and indents. |

20. UI Architecture Additions

The V3.0 update introduces high-performance, non-intrusive UI structures designed to float gracefully over any active application without stealing active system focus.

Floating Explanation HUD

[Swift Code]

struct HUD: View { ... }

■ High-Confidence Findings

- The code maps a payment verification loop.

▨ Probable Interpretations

- 'gateway' likely resolves to a networking client.

□ Missing Context

- 'Order' type definition is missing.

20.1 Floating Explanation HUD

- **Host Window Structure:** Engineered as a custom NSPanel configured with style masks `.nonActivatingPanel` and `.utilityWindow`. This ensures it behaves as an overlay window that is dismissible, floats on top of other content, and accepts mouse interactions without forcing the user out of their primary editing application.
- **Dynamic SwiftUI Viewport:** Renders the AI streaming response utilizing a highly tailored markdown engine, complete with copy-to-clipboard elements, syntax highlighting for

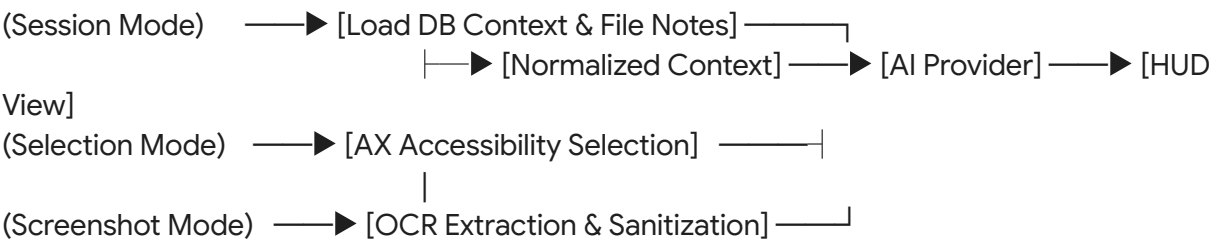
code blocks, and clear category division tabs.

20.2 Selection Overlay (Screenshot Mode)

- **Visual Presentation:** A fullscreen, transparent black overlay panel (NSWindow.Level.statusBar) that appears on the screen when visual capture begins.
- **Interaction Paradigm:** Updates the mouse pointer to a precise crosshair, allowing the user to select an exact region on screen. Displays a real-time, semi-transparent cropping rectangle while dragging.

21. Unified Explanation Pipeline

While each mode acquires context through different channels, all inputs converge into a singular, cohesive prompt consolidation and explanation pipeline.

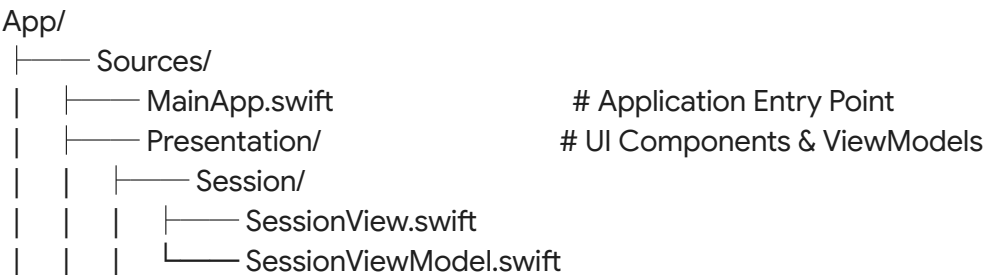


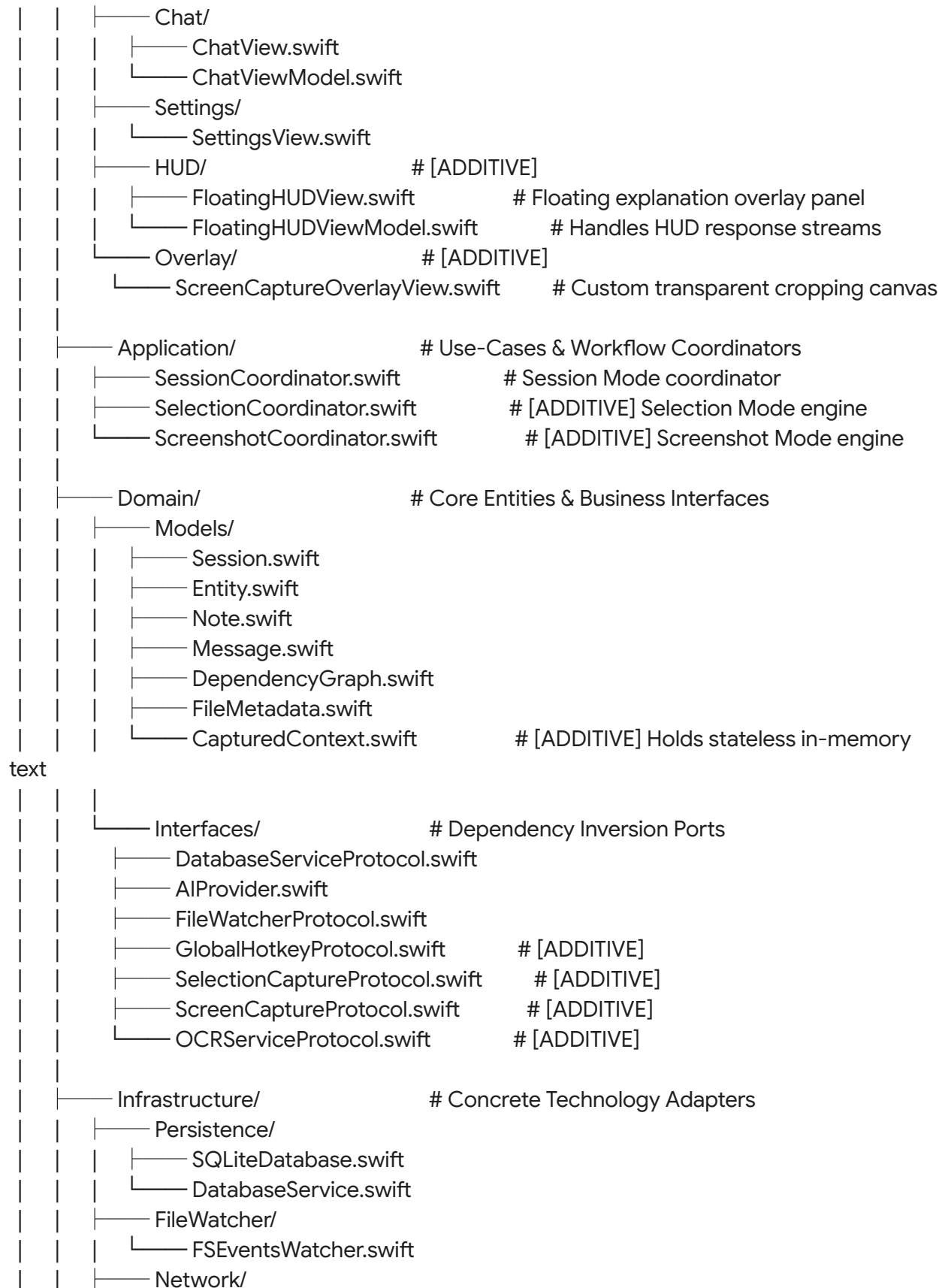
Context Acquisition Comparison

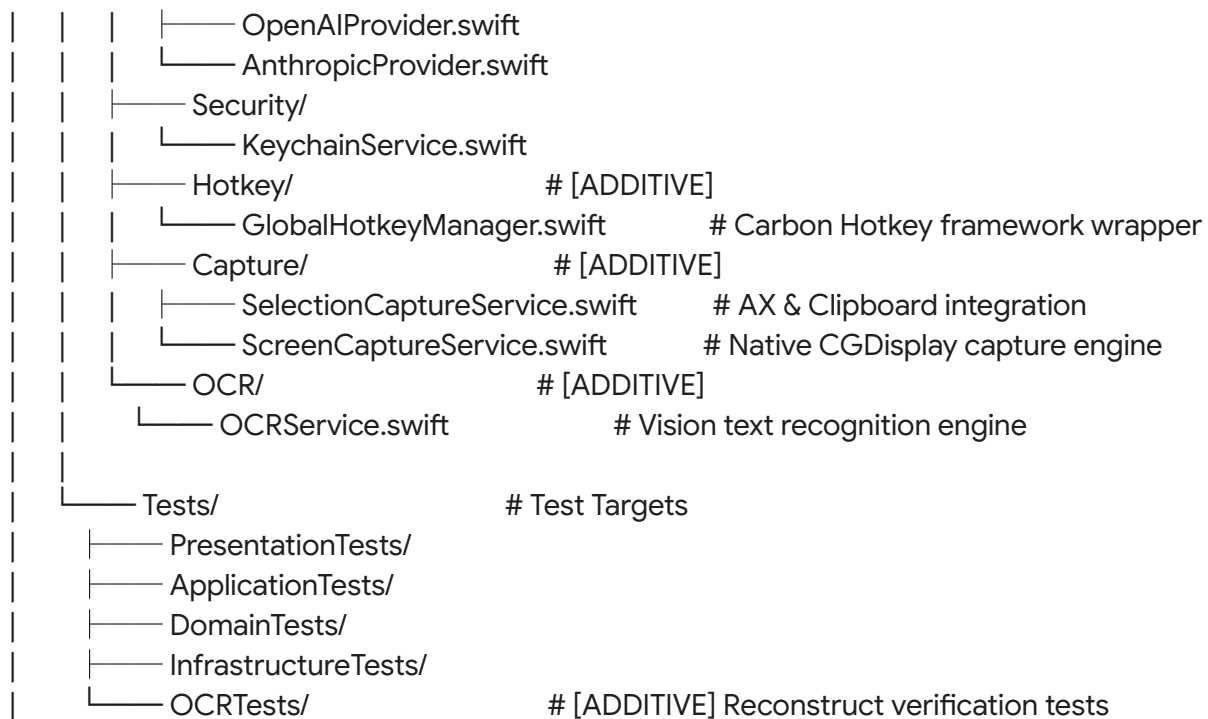
- **Session Mode:** Pulls deeply integrated context from the local SQLite database, including surrounding entity files, dependency call chains, global file summaries, and active chat threads.
- **Selection Mode:** Operates on the captured text buffer. It bypasses session state and inserts the *Incomplete Context Strategy* prompts directly into the compiler engine.
- **Screenshot Mode:** Translates the visual image slice into structural text using the native OCRService, normalizes syntax bugs, and applies the *Incomplete Context Strategy* prompts.

22. Physical Project Directory Structure

New files introduced in the V3.0 architecture update are highlighted with [ADDITIVE]. Unaltered Session Mode files remain untouched.







23. Dependency Injection Design

All infrastructure components are resolved in the unified AppContainer registration layer upon application initialization.

@MainActor

```
final class AppContainer: ObservableObject {
    // Session Mode Infrastructure Ports
    let database: DatabaseServiceProtocol
    let fileWatcher: FileWatcherProtocol
    let sessionService: SessionUseCase
    let chatService: ChatUseCase
    let aiProvider: AIProvider
    let keychain: KeychainService

    // [ADDITIVE] Selection & Screenshot Mode Ports
    let hotkeyManager: GlobalHotkeyProtocol
    let selectionCapture: SelectionCaptureProtocol
    let screenCapture: ScreenCaptureProtocol
    let ocrService: OCRServiceProtocol
    let selectionCoordinator: SelectionCoordinator
    let screenshotCoordinator: ScreenshotCoordinator
}
```

```

init() {
    // 1. Initialize Core Security & AI
    self.keychain = KeychainService()
    let dbService = SQLiteDatabase.bootstrap()
    self.database = dbService

    let apiKey = (try? keychain.getAPIKey(for: .openai)) ?? ""
    let resolvedAIProvider = OpenAIProvider(apiKey: apiKey)
    self.aiProvider = resolvedAIProvider

    // 2. Initialize Session Mode Adapters
    self.fileWatcher = FSEventsWatcher()
    self.sessionService = SessionService(database: dbService)
    self.chatService = ChatService(database: dbService, provider: resolvedAIProvider)

    // 3. Initialize [ADDITIVE] Subsystems
    self.selectionCapture = SelectionCaptureService()
    self.screenCapture = ScreenCaptureService()
    self.ocrService = OCRService()
    self.hotkeyManager = GlobalHotkeyManager()

    // 4. Register [ADDITIVE] Application Coordinators
    self.selectionCoordinator = SelectionCoordinator(
        capture: self.selectionCapture,
        ai: resolvedAIProvider
    )
    self.screenshotCoordinator = ScreenshotCoordinator(
        capture: self.screenCapture,
        ocr: self.ocrService,
        ai: resolvedAIProvider
    )

    // 5. Bind Unified Hotkey Actions
    self.hotkeyManager.registerHotkey(keyCombo: "\Space") { [weak self] in
        self?.routeUnifiedHotkeyTrigger()
    }
}

/// Executes the unified routing heuristic to choose between Selection and Screenshot Mode
private func routeUnifiedHotkeyTrigger() {
    Task {
        // Check Accessibility APIs for an active text selection
        if let selectedText = await selectionCapture.retrieveActiveSelection(timeout: 0.15) {

```

```

        // If text was successfully resolved, route to Selection Mode
        await selectionCoordinator.explainSelection(selectedText)
    } else {
        // Otherwise, fall back to capturing screen visual coordinates
        await screenshotCoordinator.captureAndExplain()
    }
}
}
}
}
}

```

24. Strategic Future Evolution Paths

This unified architecture scales to support the following upcoming features without breaking backwards compatibility:

- **Vector-Driven Semantic Workspace Mapping:** The in-memory text buffers resolved from Selection and Screenshot mode can easily be fed into local text-embedding models and queried against regional index databases, integrating on-the-fly search.
- **On-the-Fly Refactoring Patches:** Selection Mode can extend its context pipeline to draft standalone refactoring snippets, passing patches to the user's active application clipboard via synthesized key configurations.

25. MVP Implementation Order (Updated Roadmap)

To maintain stability, the engineering workflow executes sequentially. The newly integrated modes are **prepended** to the original implementation roadmap, allowing the development team to validate basic API networking and visual UI frameworks before deploying the complex database migrations required for Session Mode.

PHASE 1: Selection Mode Core [NEW]

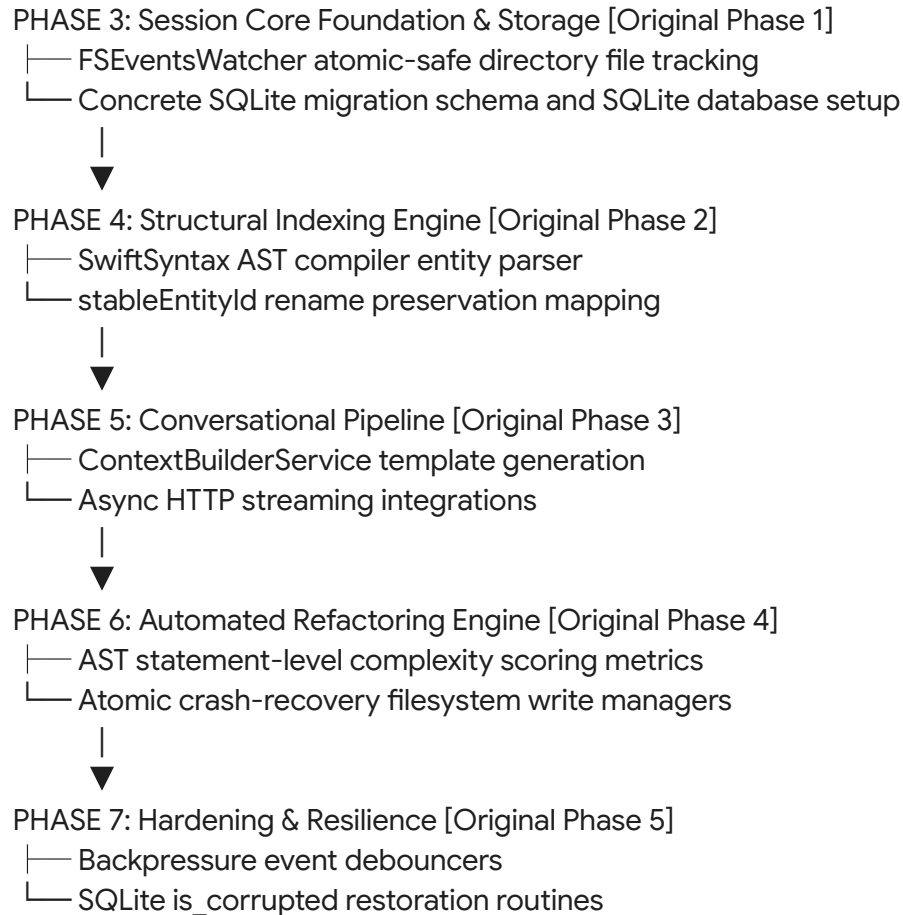
- ├─ GlobalHotkeyManager Carbon hooks integration
- ├─ SelectionCaptureService Accessibility focus querying
- ├─ Floating HUD window setup (non-activating NSPanel)
- └─ Incomplete Context prompt implementation



PHASE 2: Screenshot Mode & OCR [NEW]

- ├─ Screen capture overlay dragging canvas UI
- ├─ VNRecognizeTextRequest accurately-tuned API integrations
- ├─ Indentations and symbol normalization heuristics
- └─ Integrated fallback routing framework





25.1 Phase 1: Selection Mode Core (New Phase)

- **Targets:** Build the hotkey registration, accessibility integrations, and the floating overlay UI panels.
- **Deliverables:**
 - Low-level GlobalHotkeyManager binding system-wide activation keys.
 - SelectionCaptureService capturing active text selections with a keyboard clipboard-fallback loop.
 - Stateless FloatingHUDView displaying floating explainers without taking window focus.
 - Dynamic prompt patterns managing the *Incomplete Context Strategy*.

25.2 Phase 2: Screenshot Mode & OCR (New Phase)

- **Targets:** Build visual crop overlays, native Vision OCR execution layers, and normalized token sanitizers.
- **Deliverables:**
 - Fullscreen translucent cropping overlays displaying tracking crosshair coordinates.
 - OCRService utilizing accurate native Vision API layers without standard dictionary corrections.

- Deterministic parsing cleanups reconstructing block indentations, line wraps, and syntactic brackets.
- Single-hotkey routing heuristics multiplexing between selections and screenshot crops.

25.3 Phase 3: Session Core Foundation & Storage (Original Phase 1)

- **Targets:** Build the local database, storage paths, and directory file watch integrations.
- **Deliverables:**
 - Migration scripts initializing sessions, entities, entity_dependencies, and messages tables.
 - Native FSEvents directory monitors filtering updates to bypass editor atomic save unlinks.
 - Security-scoped bookmark resolvers maintaining sandbox read/write file access across restarts.

25.4 Phase 4: Structural Indexing Engine (Original Phase 2)

- **Targets:** Build compiler-supported AST code structural analysis parsers.
- **Deliverables:**
 - Native SwiftSyntax parsers extracting code structures (functions, methods, classes).
 - Auto-populating local dependency junction schemas.
 - Structural similarity heuristics ($S \geq 0.85$) mapping symbol renames to preserve user notes histories.

25.5 Phase 5: Conversational Pipeline (Original Phase 3)

- **Targets:** Core chat and contextual explanation features.
- **Deliverables:**
 - Centralized ContextBuilderService managing context-gathering operations.
 - Prompt compilers prioritizing local notes and summaries to conserve tokens.
 - AI streaming connections featuring exponential backoff retries.

25.6 Phase 6: Automated Refactoring Engine (Original Phase 4)

- **Targets:** Code refactoring and transaction handlers.
- **Deliverables:**
 - Complexity priority scoring engines measuring AST statement counts and nesting depths.
 - Automated disk modification writers applying LLM patches.
 - Memory backup transactions protecting against stream interrupts.

25.7 Phase 7: Hardening & Resilience (Original Phase 5)

- **Targets:** Network, token budget, and database recovery testing.
- **Deliverables:**
 - Debouncing algorithms limiting execution frequencies.

- Token mitigation strategies keeping massive files within context window budgets.
- SQLite session recovery loops rebuilding broken records.